



数据结构与算法分析-5



人工智能与自动化学院

韩守东

shoudonghan@hust.edu.cn

13720301586



上节内容回顾

- 串的顺序存储：定长顺序存储和堆分配存储
- 插入和删除操作与线性表相同
- 朴素字符串匹配：需要同时回溯主串和子串的指针，最坏情况复杂度为 $O(mn)$

第1趟

S	a	b	<u>b</u>	a	b	a
T	a	b	a			

$i=3$
 $j=3$

第2趟

S	a	<u>b</u>	b	a	b	a
T		a	b	a		

$i=2$
 $j=1$

- KMP匹配算法：S的i不动，仅移动T的j
 - 在子串j处的失配利用了 $1 \dots j-1$ 处字符串的信息
 - 根据子串可以预先判断j处失配时，应该选 $\text{next}[j]=k$ 与主串i处比较。因 $\text{next}[1]=0$ 可知，假设 $\text{next}[1] \dots \text{next}[j]$ 已知，求 $\text{next}[j+1]$
 - 递推思想：(1) $x=k$, (2)比较 $T[j]$ 与 $T[x]$,若相等则 $\text{next}[j+1]=x+1$ 否则找 $x=\text{next}[x]$,返回(2);

若(2)循环未找到 $T[j]$ 与 $T[x]$ 相等，则 $\text{next}[j+1]=1$.



5. 数组和广义表

- ▶ 5.1 数组的定义
- ▶ 5.2 数组的顺序存储结构
- ▶ 5.3 矩阵的压缩存储
- ▶ 5.4 广义表的定义
- ▶ 5.5 广义表的存储结构



5.1 数组的定义

- ▶ 数组可以看成是一种特殊的线性表，即线性表中数据元素本身也是一个线性表

- ▶ 定义

$$A_{m \times n} = \begin{bmatrix} (a_{11} & a_{12} & \dots & \dots & a_{1n}) \\ (a_{21} & a_{22} & \dots & \dots & a_{2n}) \\ (\dots & \dots & \dots & \dots & \dots) \\ (a_{m1} & a_{m2} & \dots & \dots & a_{mn}) \end{bmatrix}$$

- 数组特点

- ✧ 数组结构固定

- ✧ 数据元素同构

- 数组运算

- ✧ 给定一组下标，存取相应的数据元素

- ✧ 给定一组下标，修改数据元素的值



5. 2数组的顺序存储结构

▶ 次序约定

- ▶ 以行序为主序
- ▶ 以列序为主序

a_{11}	a_{12}		a_{1n}
a_{21}	a_{22}		a_{2n}
.....			
a_{m1}	a_{m2}		a_{mn}

a_{11}
a_{12}
...
a_{1n}
a_{21}
a_{22}
...
a_{2n}
...
a_{n1}
a_{m2}
...
a_{nn}

a_{11}
a_{21}
...
a_{m1}
a_{12}
a_{22}
...
a_{m2}
...
a_{1n}
a_{2n}
...
a_{mn}

列序: $Loc(a_{ij}) = Loc(a_{11}) + [(j-1)m + (i-1)] * l$



5.3 矩阵的压缩存储

- ▶ 对称矩阵
- ▶ 三角矩阵
- ▶ 对角矩阵
- ▶ 稀疏矩阵

稀疏矩阵的压缩存储方法



对称矩阵

$$\begin{bmatrix} \mathbf{a}_{11} & \mathbf{a}_{12} & \dots & \dots & \dots & \mathbf{a}_{1n} \\ \mathbf{a}_{21} & \mathbf{a}_{22} & \dots & \dots & \dots & \mathbf{a}_{2n} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \mathbf{a}_{n1} & \mathbf{a}_{n2} & \dots & \dots & \dots & \mathbf{a}_{nn} \end{bmatrix}$$

按行序为主序：

a ₁₁	a ₂₁	a ₂₂	a ₃₁	a ₃₂		a _{n1}		a _{nn}
k=0	1	2	3	4		n(n-1)/2		n(n+1)/2-1

$$k = \begin{cases} i(i-1)/2 + j-1, & i \geq j \\ j(j-1)/2 + i-1, & i < j \end{cases}$$



三角矩阵

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 \\ a_{21} & a_{22} & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & 0 \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

按行序为主序：

a_{11}	a_{21}	a_{22}	a_{31}	a_{32}		a_{n1}		a_{nn}
$k=0$	1	2	3	4		$n(n-1)/2$		$n(n+1)/2-1$

$$\text{Loc}(a_{ij}) = \text{Loc}(a_{11}) + [i(i-1)/2 + (j-1)] * l, \quad l \text{ 为元素单元大小}$$



对角矩阵

$$\begin{bmatrix}
 a_{11} & a_{12} & 0 & \dots & \dots & 0 \\
 a_{21} & a_{22} & a_{23} & 0 & \dots & 0 \\
 0 & a_{32} & a_{33} & a_{34} & 0 & 0 \\
 \dots & \dots & \dots & \dots & \dots & \dots \\
 0 & 0 & \dots & a_{n-1,n-2} & a_{n-1,n-1} & a_{n-1,n} \\
 0 & 0 & \dots & \dots & a_{n,n-1} & a_{nn}
 \end{bmatrix}$$

按行序为主序：

a ₁₁	a ₁₂	a ₂₁	a ₂₂	a ₂₃			a _{n(n-1)}	a _{nn}
k=0	1	2	3	4				3n-3

$$\text{Loc}(a_{ij}) = \text{Loc}(a_{11}) + [2(i-1) + (j-1)] * 1$$



稀疏矩阵

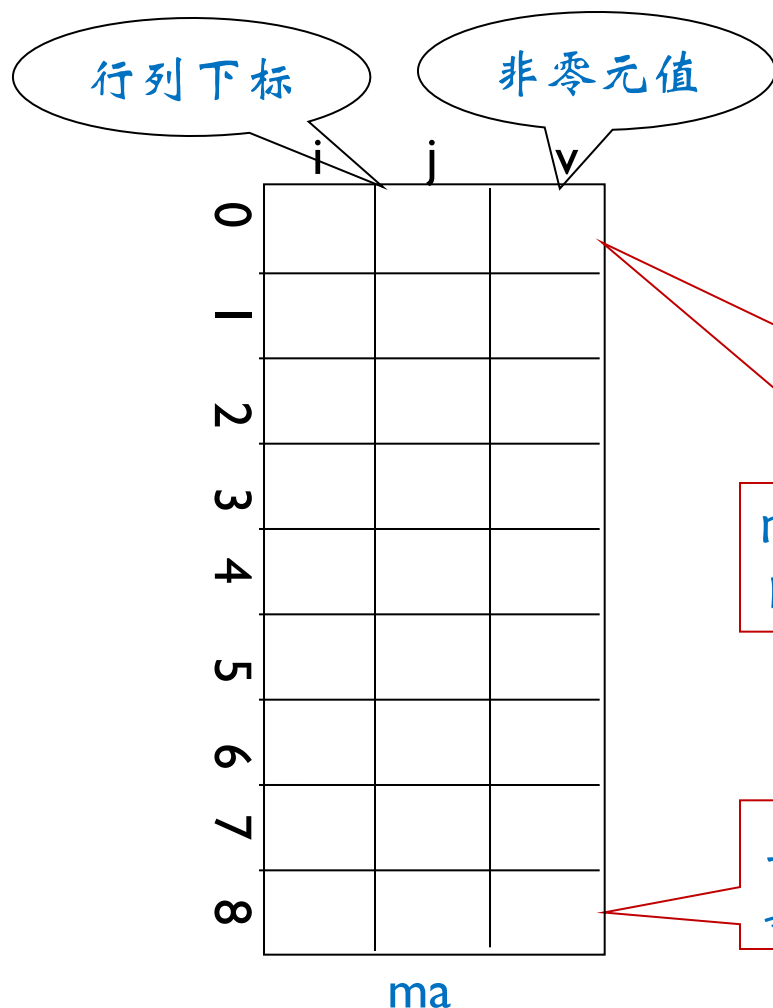
- ✧ 定义：非零元较零元少，且分布没有一定规律的矩阵
- ✧ 压缩存储原则：只存矩阵的行列维数和每个非零元的行列下标及其值

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

M由{(1,2,12), (1,3,9), (3,1,-3), (3,6,14), (4,3,24),
(5,2,18), (6,1,15), (6,4,-7)} 和矩阵维数(6,7)唯一确定



稀疏矩阵的压缩存储方法



$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

ma[0].i,ma[0].j,ma[0].v分别存放矩阵行列维数和非零元个数

三元组表所需存储单元个数为 $3(t+1)$
其中 t 为非零元个数



三元组顺序表

```
#define MAXSIZE 12500
typedef struct {
    int i, j;
    ElemType e;
} Triple;
typedef struct {
    Triple data[MAXSIZE+1];
    int mu, nu, tu;
}
```



带辅助行向量的二元组表

增加一个辅助数组NRA[m+1]，其物理意义是第i行第一个非零元在二元组表中的起始地址（m为行数）

显然有：
$$\begin{cases} \text{NRA}[1]=1 \\ \text{NRA}[i]=\text{NRA}[i-1]+\text{第}i\text{-1行非零元个数}(i\geq 2) \end{cases}$$

列下标和
非零元值

NRA[0]不用或
存矩阵行数

	NRA	j	v
0	6	7	8
1	1	2	12
2	3	3	9
3	3	1	-3
4	5	6	14
5	6	3	24
6	7	2	18
		1	15
		4	-7

矩阵列数和
非零元个数

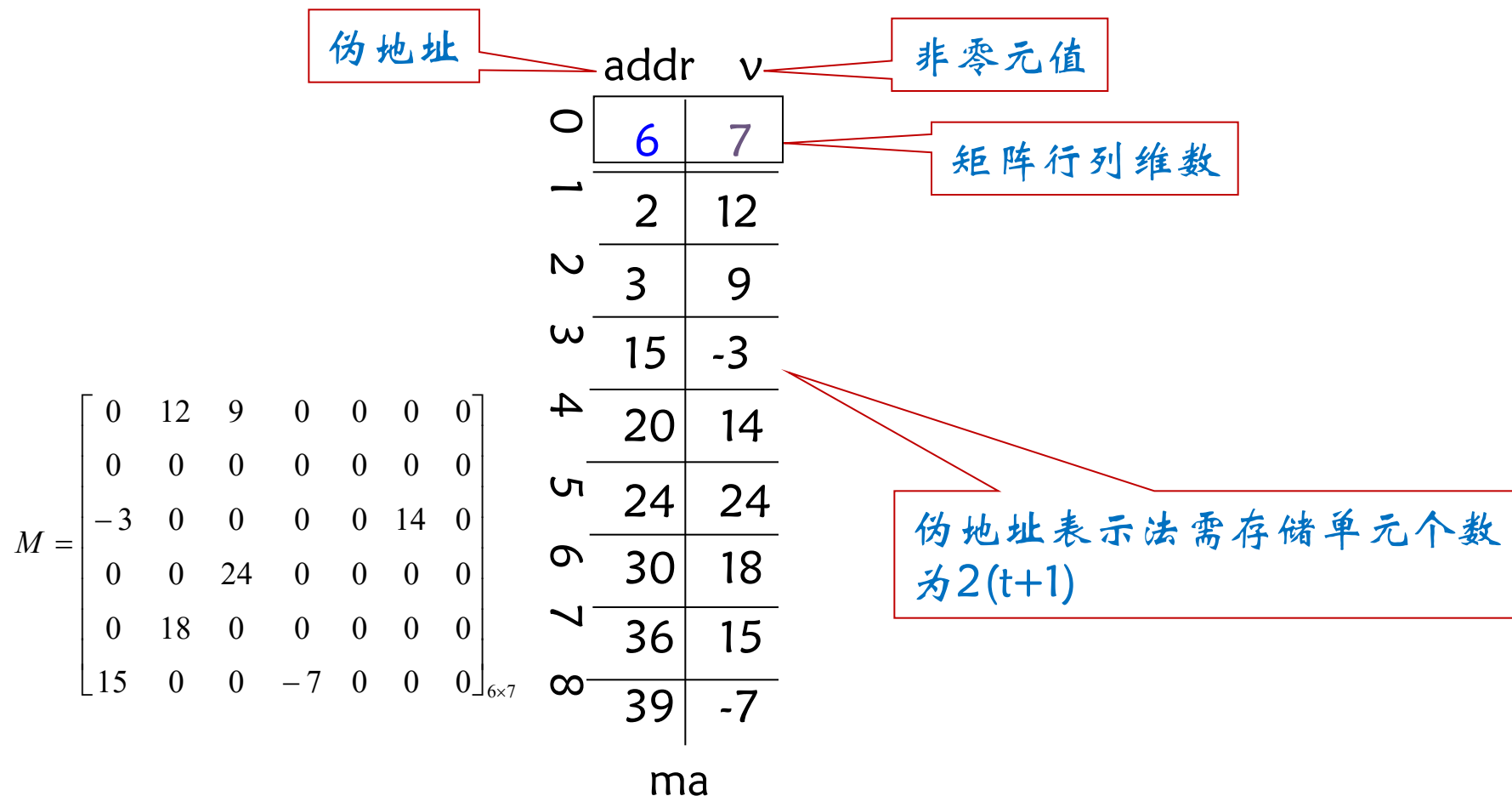
二元组表需存储单元
个数为 $2(t+1)+m+1$

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$



伪地址表示法

伪地址：本元素在矩阵中（包括零元素在内）
按行优先顺序的相对位置





求转置矩阵

☆ 问题描述：已知一个稀疏矩阵的三元组表，求该矩阵转置矩阵的三元组表

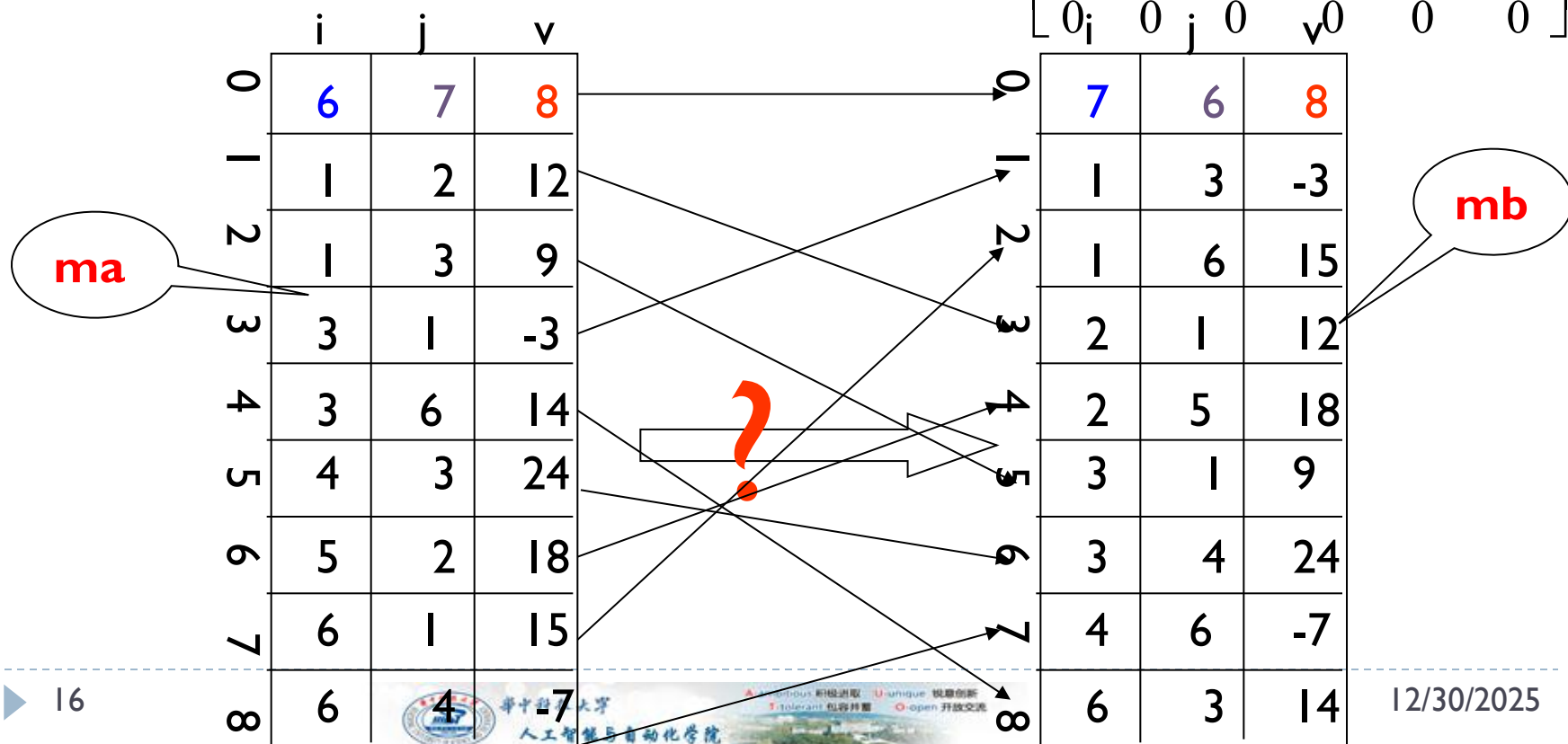
☆ 问题分析

一般矩阵转置算法：

```
for(col=0;col<n;col++)  
    for(row=0;row<m;row++)  
        n[col][row]=m[row][col];  
T(n)=O(m×n)
```

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

$$N = \begin{bmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}_{7 \times 6}$$





解法

☆解决思路：只要做到

①将矩阵行、列维数互换

②将每个三元组中的*i*和*j*相互调换

③重排三元组次序，使mb中元素以N的行(M的列)为主序

方法一：按M的列序转置

即按mb中三元组次序依次在ma中找到相应的三元组进行转置。

为找到M中每一列所有非零元素，需对其三元组表ma从第一行起扫描一遍。由于ma中以M行序为主序，所以由此得到的恰是mb中应有的顺序。

☆算法描述：

☆算法分析： $T(n) = O(M \text{的列数} n \times \text{非零元个数} t)$

☆若 t 与 $m \times n$ 同数量级，则 $T(n) = O(m \times n^2)$





算法

```
Status TransposeSMatrix(TSMatrix M, TSMatrix &T) {  
    T.mu = M.nu; T.nu = M.mu; T.tu = M.tu;  
    if(T.tu) {  
        q=1;  
        for(col=1; col<=M.nu; ++col)  
            for(p=1; p<=M.tu; ++p)  
                if(M.data[p].j == col) {  
                    T.data[q].i = M.data[p].j;           T.data[q].j =  
                    M.data[p].i;                           M.data[p].i;  
                    T.data[q].e = M.data[p].e; ++q; }  
                }  
    }  
    return OK;  
}
```



过程示意

	i	j	v	
0	6	7	8	
p → 1	1	2	12	← p
p → 2	1	3	9	← p
p → 3	3	1	-3	← p
p → 4	3	6	14	← p
p → 5	4	3	24	← p
p → 6	5	2	18	← p
p → 7	6	1	15	← p
p → 8	6	4	-7	← p

ma

col=1

	i	j	v	
0	7	6	8	
q → 1	1	3	-3	
q → 2	1	6	15	
q → 3	2	1	12	
q → 4	2	5	18	
q → 5	3	1	9	
6	3	4	24	
7	4	6	-7	
8	6	3	14	

mb

col=2



方法二：快速转置

- ▶ 即按 **ma** 中三元组次序转置，转置结果放入 **b** 中恰当位置。
此法关键是要预先确定 **M** 中每一列第一个非零元在 **mb** 中位置，为确定这些位置，转置前应先求得 **M** 的每一列中非零元个数。

- ▶ 实现：设两个数组

$\text{num}[\text{col}]$ ：表示矩阵 **M** 中第 col 列中非零元个数

$\text{cpot}[\text{col}]$ ：指示 **M** 中第 col 列第一个非零元在 **mb** 中位置

- ▶ 显然有：
$$\begin{cases} \text{cpot}[1]=1; \\ \text{cpot}[\text{col}]=\text{cpot}[\text{col}-1]+\text{num}[\text{col}-1]; \end{cases} \quad (2 \leq \text{col} \leq \text{ma}[0].j)$$

$$M = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix}_{6 \times 7}$$

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9



算法描述

```
Status FastTransposeSMatrix(TSMatrix M, TSMatrix &T) {
    T.mu = M.nu; T.nu = M.mu; T.tu = M.tu;
    if(T.tu) {
        for(col=1; col<=M.nu; ++col) num[col]=0; //初始化
        for(t=1; t<=M.tu, ++t) ++num[M.data[t].j]; //统计每列的非零元素个数
        cpot[1] = 1;
        /*计算每列的首个非零元位置*/
        for(col=2; col<=M.nu; ++col)
            cpot[col] = cpot[col-1] + num[col-1];
        for(p=1; p<=M.tu; ++p) { //扫描M三元组表, 将每个元素填入对应的T表位置
            col = M.data[p].j; q = cpot[col];
            T.data[q].i = M.data[p].j; T.data[q].j = M.data[p].i;
            T.data[q].e = M.data[p].e; ++cpot[col]; }
    }
    return OK;
}
```

$T(n) = O(M \text{的列数} n + \text{非零元个数} t)$ 若 t 与 $m \times n$ 同数量级, 则 $T(n) = O(m \times n)$



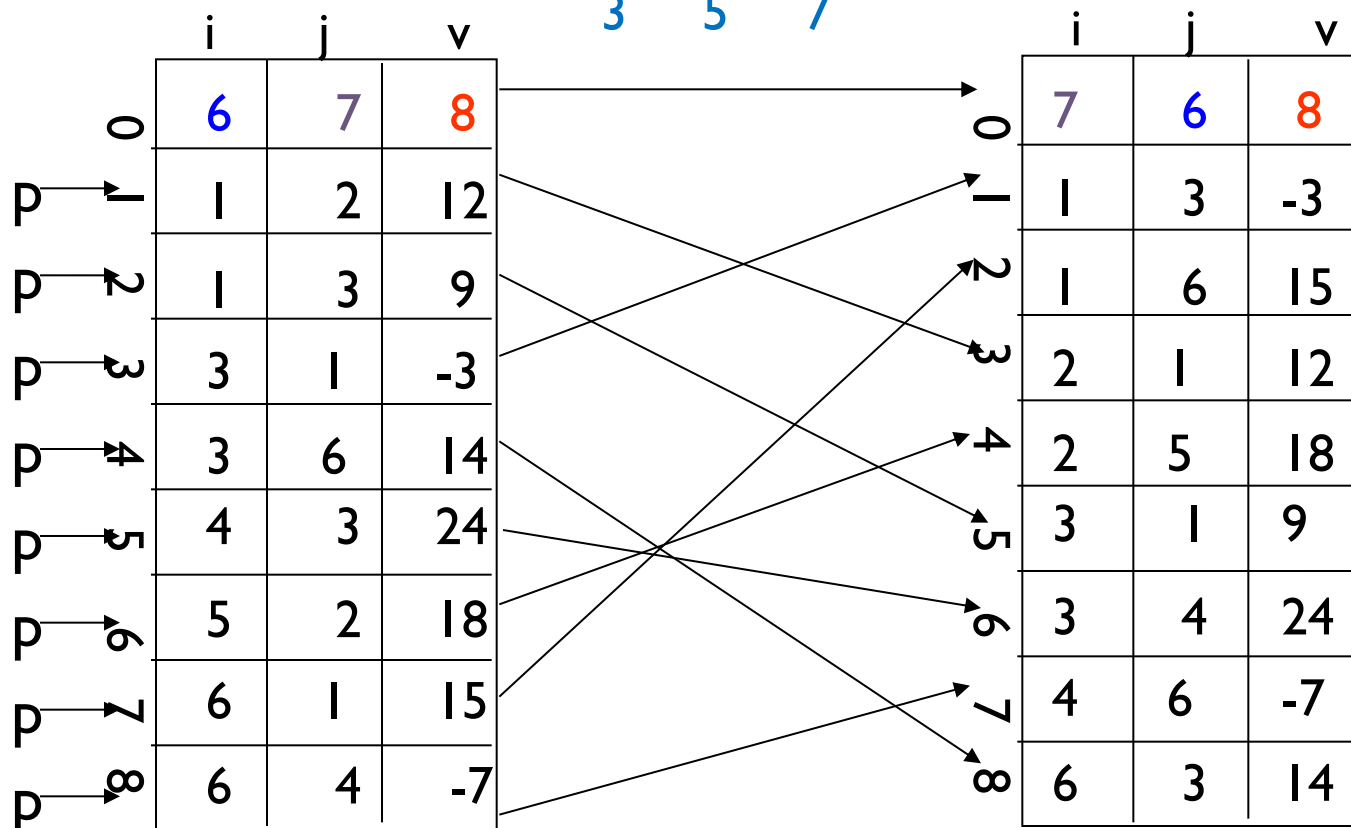
统计表

转置矩阵b每行的非零元个数

转置矩阵b每行首个非零元位置

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9

2 4 6 9
3 5 7



ma

mb



链式存储结构

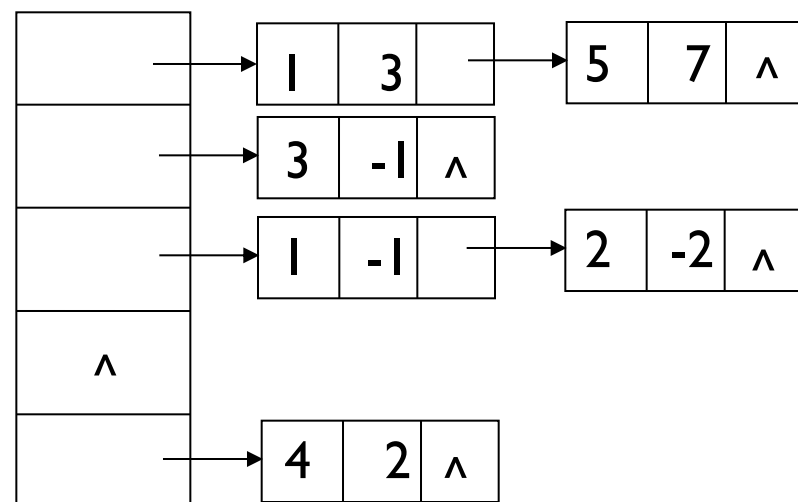
□ 带行指针向量的单链表表示

☆ 每行的非零元用一个单链表存放

☆ 设置一个行指针数组，指向本行第一个非零元结点；若本行无非零元，则指针为空

```
typedef struct node
{
    int col;
    int val;
    struct node *link;
}D;
typedef struct node *TD;
```

$$A = \begin{bmatrix} 3 & 0 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 & 0 \\ -1 & -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \end{bmatrix}$$



需存储单元个数为 $3t+m$



十字链表

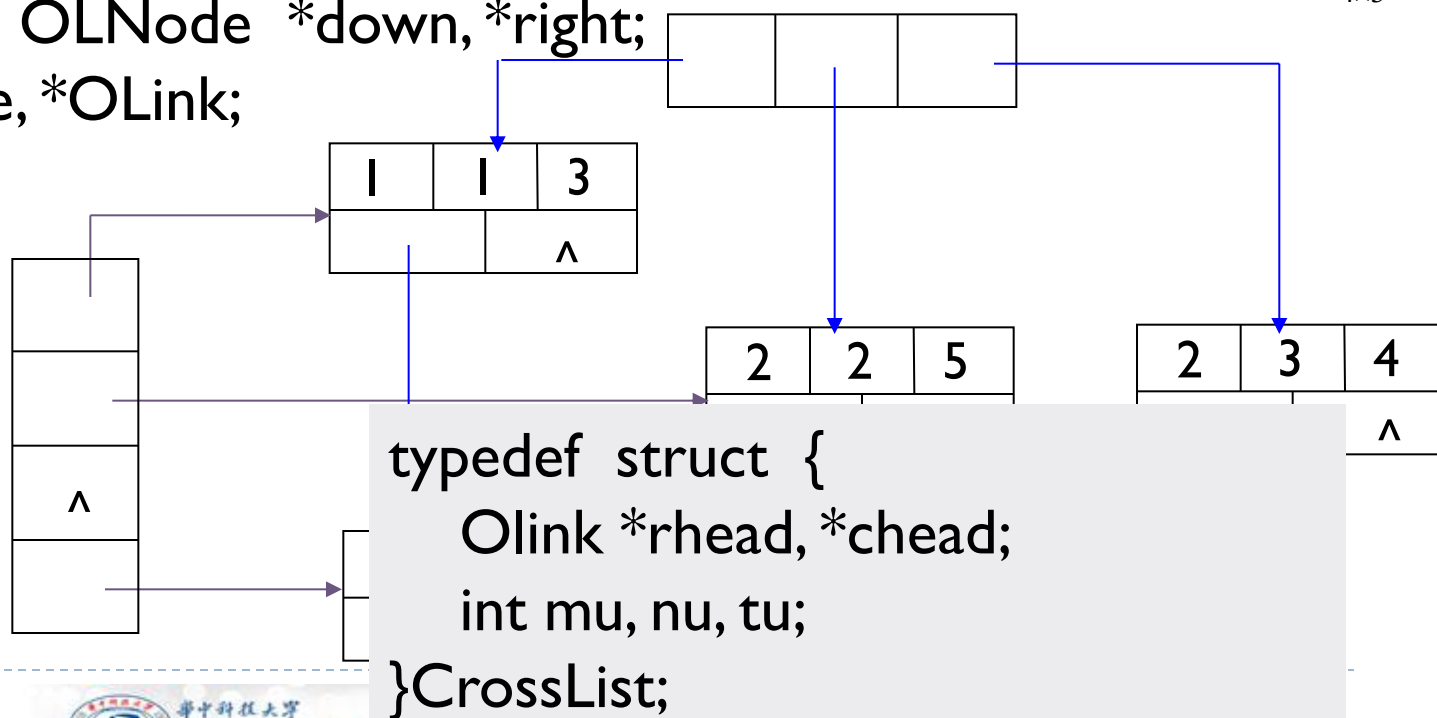
☆ 设行指针数组和列指针数组，分别指向每行、列第一个非零元

☆ 结点定义

```
typedef struct OLNode {  
    int i, j;  
    ElemType e;  
    struct OLNode *down, *right;  
} OLNode, *OLink;
```

row	col	val
down	right	

$$A = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 4 \\ 0 & 0 & 0 \\ 8 & 0 & 0 \end{bmatrix}_{4 \times 3}$$





建立十字链表算法

☆ 从键盘接收信息

▶ 见书P104：算法5.4

▶ 任意的非零元输入先后次序

▶ 注意：行表和列表的插入操作

☆ 算法分析： $T(n)=O(t \times s)$

其中： t —非零元个数

$$s = \max(m, n)$$



建立十字链表算法代码

```
Status CreateSMatrix_OL(CrossList &M) {  
    //创建系数矩阵M,采用十字链表存储表示  
    if(M) free(M);  
    scanf(&m,&n,&t); // 输入M的行数、列数和非零元个数  
    M.mu = m; M.nu = n; M.tu = t;  
    //初始化行列头指针数组  
    if( !(M.rhead = (Olink*)malloc((m+1)* sizeof (Olink))) ) exit (OVERFLOW);  
    if( !(M.thead = (Olink*)malloc((n+1)* sizeof (Olink))) ) exit (OVERFLOW);  
    M.rhead[] = M.thead[] = NULL; //置行列链表为空链表  
    for(scanf(&i,&j,&e); i!=0; scanf(&i,&j,&e)) { //按任意次序输入非零元  
        if(!p=(OLNode*)malloc(sizeof(OLNode))) exit(OVERFLOW);  
        p->i=i; p->j=j; p->e=e;           // 生成结点  
        //若p为i行链表中的首节点  
        if(M.rhead[i] == NULL || M.rhead[i]->j > j) {p->right = M.rhead[i]; M.rhead[i]=p;}  
        else { // 寻找结点在行表中的插入位置  
            for(q=M.rhead[i]; (q->right) && q->right->j < j; q=q->right);  
            p->right=q->right; q->right=p;} //完成行插入  
    }  
}
```



建立十字链表算法代码

```
//若p为j列链表中的首节点
if(M.chead[j] == NULL || M.rhead[j]->i>i) {p->down = M.chead[j]; M.rhead[j]=p;}
else{// 寻找结点在列表中的插入位置
    for(q=M.chead[j]; (q->down) && q->down->i<i; q=q->down);
    p->down=q->down; q->down=p;} //完成列插入
}
return OK
} //CreateSmatrix_OL
```



例子

$m=4, n=3$

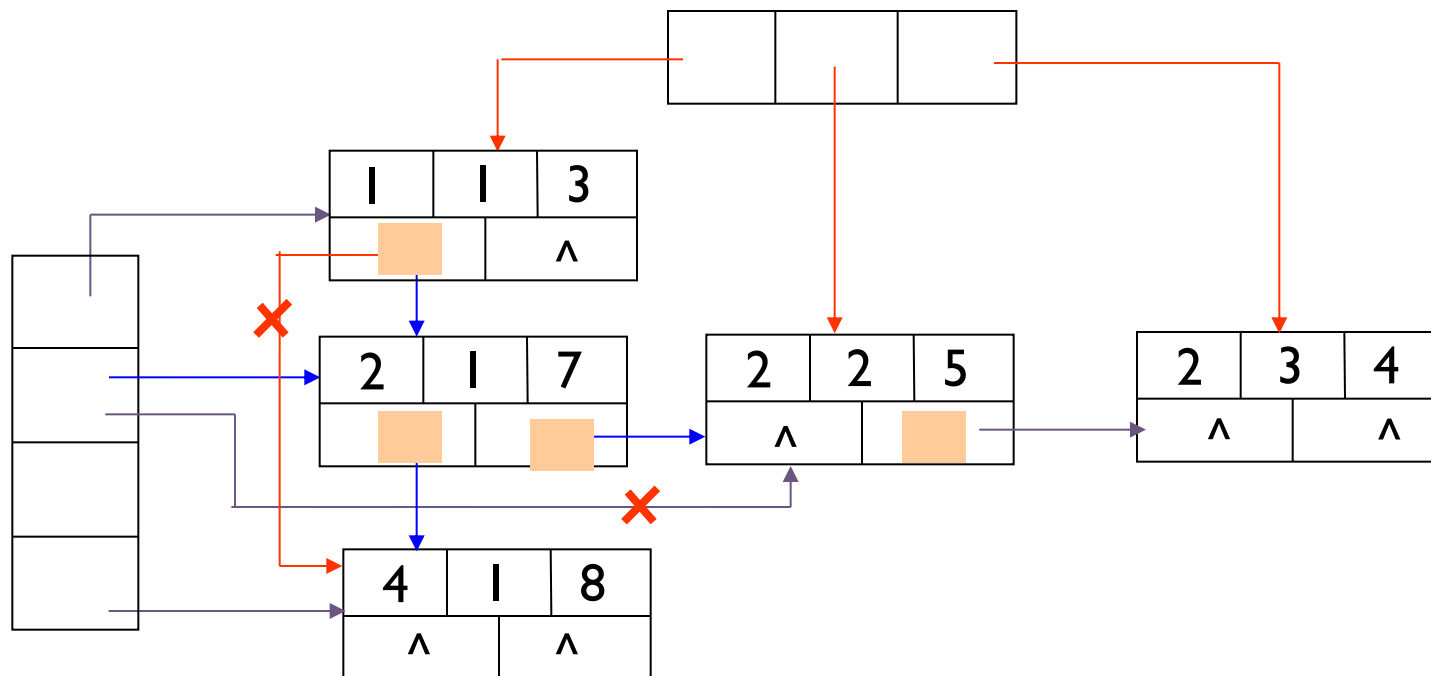
1,1,3

2,2,5

2,3,4

4,1,8

2,1,7





小结

- ▶ 数组是一类特殊的线性结构，无删除和插入操作
- ▶ 特殊矩阵：特殊的存储处理提高其存储效率
- ▶ 稀疏矩阵：仅保存非零数据
 - 三元组、行数组链表，十字链表表示
 - 不同表示方式下的操作特殊性



5.4 广义表的定义

- ▶ 广义表是线性表的推广，也有人称之为列表(Lists用复数形式以示与统称的表list的区别)。
- ▶ LISP语言，把广义表作为基本的数据结构，就连程序也表示为一系列的广义表。
- ▶ 广义表记作 $LS=(a_1, a_2, \dots, a_n)$

其中,LS是广义表 $(a_1, a_2, \dots, a_n)$ 的名称,n是它的长度。

在线性表的定义中， a_i ($1 \leq i \leq n$)只限于单个元素。在广义表的定义中， a_i 可以是单个元素，也可以是广义表，分别称为广义表LS的原子和子表。



- 原子子表
- $LS = (\text{'c'}, (973, 863, 985, \text{'x'}), \dots)$
- 表头表尾



广义表的定义

- ▶ 广义表的定义是一个递归的定义，因为在描述广义表时又用到了广义表的概念。例子：

- $A = ()$ A为空表，长度为0
- $B = (e)$ B只有1个原子，长度为1
- $C = (a, (b, c, d))$ C长度为2，包含原子a和子表(b, c, d)
- $D = (A, B, C)$ D长度为3，代入后 $D = ((), (e), (a, (b, c, d)))$
- $E = (a, E)$ 递归表，长度为2，实为无限列表
 $E = (a, (a, (a, \dots)))$



广义表表头和表尾

- 当广义表LS非空时，称第一个元素 a_1 为LS的表头(Head)，称其余元素组成的表为LS的表尾(Tail)
- 根据前述对表头、表尾的定义可知：任何一个非空列表其表头可能是原子，也可能是列表，而其表尾必定为列表。

- $A = ()$ A 为空表，表头表尾不存在
- $B = (e)$ 表头为 e ，表尾为 $()$
- $C = (a, (b, c, d))$ 表头为 a ，表尾为 $((b, c, d))$
- $D = (A, B, C)$ 表头为 A ，表尾为 (B, C)

- 列表 $()$ 和 $(())$ 不同。前者为空表，长度 $n=0$ ；后者长度 $n=1$ ，可分解得到其表头、表尾均为空表 $()$ 。



广义表的特征

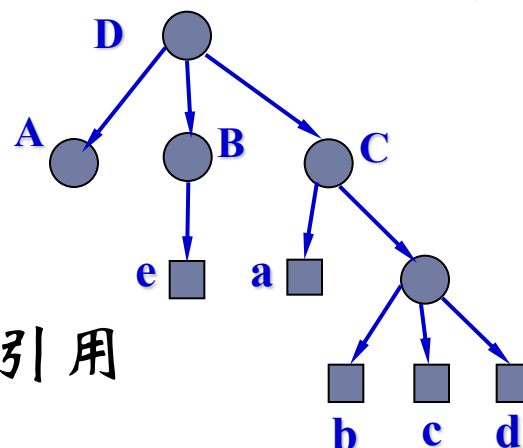
- 列表的元素可以是子表，而子表的元素还可以是子表，...。由此，列表是一个多层次的结构，可以用图形象地表示。

$A = ()$

$B = (e)$

$C = (a, (b, c, d))$

$D = (A, B, C)$



- 列表可为其他列表所共享。D可直接引用A,B,C,不必写出其实际内容

$D = (A, B, C)$

- 列表可以是一个递归的表，即列表也可以是其本身的子表

$E = (a, E)$



广义表抽象数据类型

ADT GList {

数据对象: $D = \{e_i \mid i = 1, 2, \dots, n; n \geq 0; e_i \in \text{AtomSet} \text{ 或 } e_i \in \text{GList},$

AtomSet 为某个数据对象 }
数据关系: $R_1 = \{ \langle e_{i-1}, e_i \rangle \mid e_{i-1}, e_i \in D, 2 \leq i \leq n \}$

基本操作:

InitGList(&L);

操作结果: 创建空的广义表 L。

CreateGList(&L, S);

初始条件: S 是广义表的书写形式串。

操作结果: 由 S 创建广义表 L。

DestroyGList(&L);

初始条件: 广义表 L 存在。

操作结果: 销毁广义表 L。

CopyGList(&T, L);

初始条件: 广义表 L 存在。

操作结果: 由广义表 L 复制得到广义表 T。

GListLength(L);

初始条件: 广义表 L 存在。

操作结果: 求广义表 L 的长度, 即元素个数。

GListDepth(L);

初始条件: 广义表 L 存在。

操作结果: 求广义表 L 的深度。

GListEmpty(L);

初始条件: 广义表 L 存在。

操作结果: 判定广义表 L 是否为空。

GetHead(L);

初始条件: 广义表 L 存在。

操作结果: 取广义表 L 的头。

GetTail(L);

初始条件: 广义表 L 存在。

操作结果: 取广义表 L 的尾。

InsertFirst_GL(&L, e);

初始条件: 广义表 L 存在。

操作结果: 插入元素 e 作为广义表 L 的第一元素。

DeleteFirst_GL(&L, &e);

初始条件: 广义表 L 存在。

操作结果: 删除广义表 L 的第一元素, 并用 e 返回其值。



广义表的链式表示

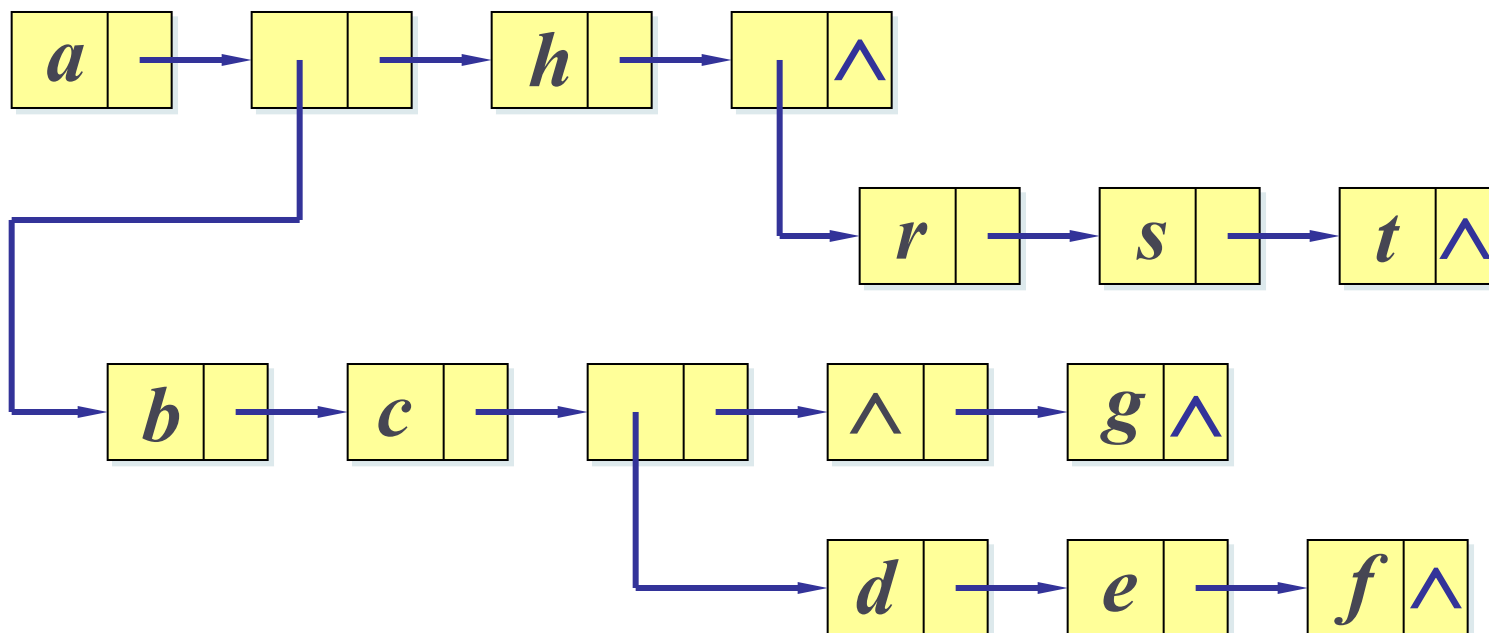
$list1 = (a, b, c, d, e)$

list1



$list2 = (a, (b, c, (d, e, f), \wedge), g), h, (r, s, t))$

list2





5.5 广义表的存储结构

- ▶ 通常采用链式存储结构
- ▶ 每个数据元素可用一个结点表示

表节点：表头和表尾

原子节点：原子

表节点表示

tag=1	hp	tp
-------	----	----

原子节点表示

tag=0	atom
-------	------

typedef enum {ATOM, LIST} ElemTag; // ATOM ==0: 原子, LIST==1:子表

typedef struct MPNode{

ElemTag tag; //区分原子结点和表结点

union{ //原子结点和表结点的联合部分

AtomType atom; //原子结点的值域

struct {struct GLNode *hp, *tp; }ptr;

//两个指针域ptr.hp和ptr.tp分别指向表头和表尾

};

} *GList; // 广义表类型

tag=1	hp	tp
-------	----	----

tag=0	atom
-------	------

- (1)除空表的表头指针为空外，对任何**非空列表**，其**表头指针**均指向一个**表结点**，且该结点中的**hp域指示列表表头**(或为原子结点，或为表结点)，**tp域指向列表表尾**(除非表尾为空，则指针为空，否则必为表结点)；
- (2)容易分清列表中原子和子表所在层次。
- (3)**最高层的表结点个数即为列表的长度**。





广义表的链式存储

► 扩展线性链表表示

表节点表示

tag=1	hp	tp
-------	----	----

原子节点表示

tag=0	atom	tp
-------	------	----

typedef enum {ATOM, LIST} ElemTag; // ATOM ==0: 原子, LIST==1:子表

typedef struct MPNode{

ElemTag tag; //区分原子结点和表结点

union{ //原子结点和表结点的联合部分

AtomType atom; //原子结点的值域

struct GLNode *hp; //表结点的表头指针

};

struct GLNode *tp; //相当于线性链表的next,指向下一个元素结点

} *GList; // 广义表类型Glist是一种扩展的线性链表



例2

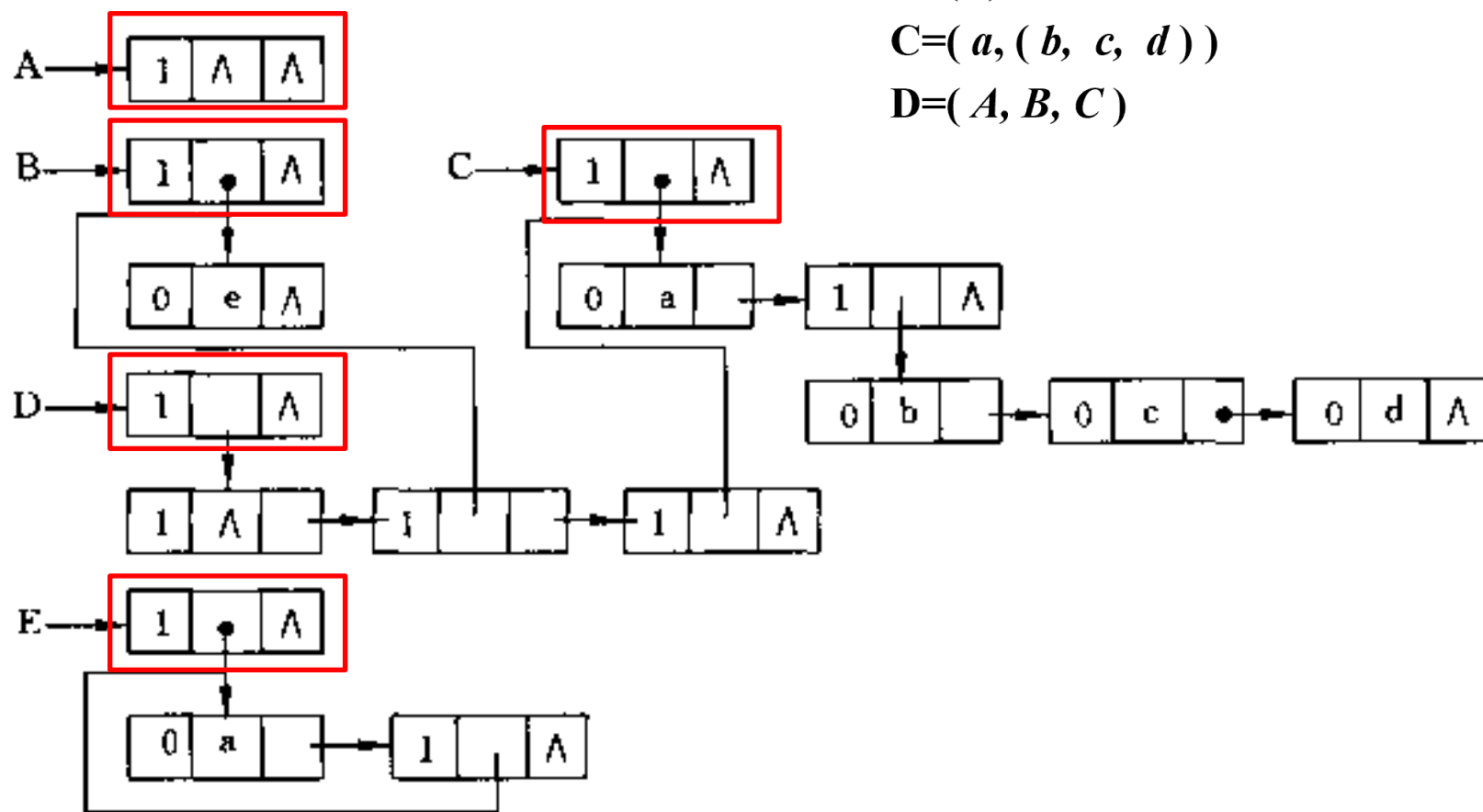
► 设表的头结点

$A = ()$

$B = (e)$

$C = (a, (b, c, d))$

$D = (A, B, C)$





5.6 例子：m元多项式表示

- ▶ 参照多项式的链式表示，即m元多项式的每项均表示成链表中的1个结点，则每项需要m+1个单元表示

- 如下式，每项需要1个系数及3个指数项表示

$$P(x, y, z) = x^{10}y^3z^2 + 2x^6y^3z^2 + 3x^5y^2z^2 + x^4y^4z + 6x^3y^4z + 2yz + 15$$

- 但若表示成

$$P(x, y, z) = \overbrace{((\underbrace{x^{10} + 2x^6}_C)y^3 + \underbrace{3x^5y^2}_D)z^2}^A + \overbrace{((\underbrace{x^4 + 6x^3}_E)y^4 + \underbrace{2y}_F)z}_{B} + 15$$

- ▶ 可将其表示成广义表形式，其深度为变量数

$$A = y((C, 3), (D, 2)) \begin{cases} C = x((1, 10), (2, 6)) \\ D = x((3, 5)) \end{cases}$$

$$P = z((A, 2), (B, 1), (15, 0))$$

$$B = y((E, 4), (F, 1)) \begin{cases} E = x((1, 4), (6, 3)) \\ F = x((2, 0)) \end{cases}$$



5.6 例子：m元多项式表示

► 采用扩展线性链表表示

- exp和coef分别为指数域和系数域
- hp和tp分别指向表头及同一层的下一结点

表节点表示

tag=1	exp	hp	tp
-------	-----	----	----

原子节点表示

tag=0	exp	coef	tp
-------	-----	------	----

```
typedef struct MPNode{
    ElemTag    tag;           //区分原子结点和表结点
    int        exp;           //指数域
    union{
        float   coef;         //系数域
        struct  MPNode *hp;    //表结点的表头指针
    };
    struct MPNode *tp;        //相当于线性链表的next,指向下一个元素结点
} *MPList;                  // m元多项式广义表类型
```



5.6 例子：m元多项式表示

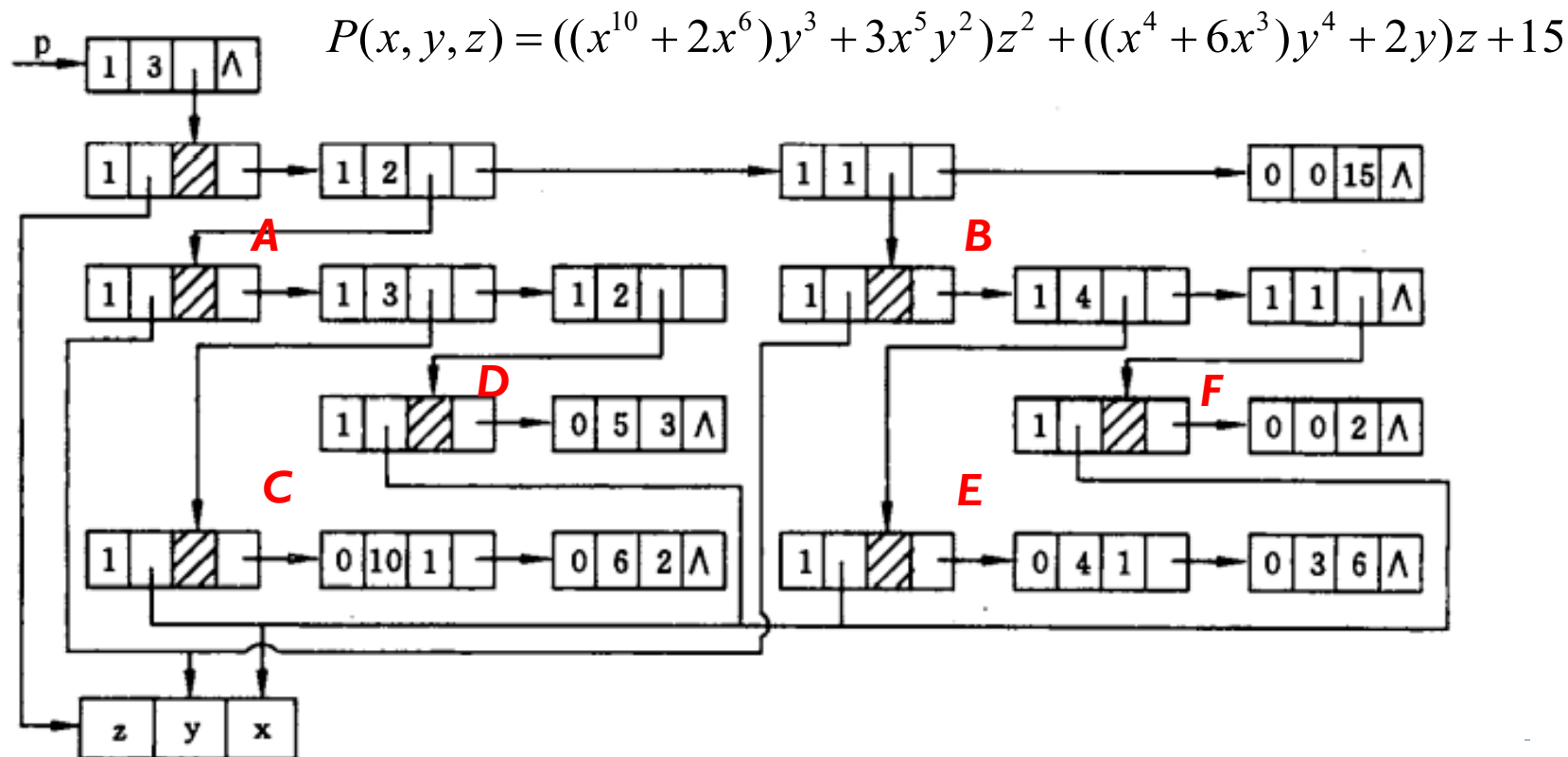
► 多项式的图结构示意图

- 最上层结点p的exp值为变元数
- 在每一层上设表头结点

$$P = z((A, 2), (B, 1), (15, 0))$$

$$A = y((C, 3), (D, 2)) \begin{cases} C = x((1, 10), (2, 6)) \\ D = x((3, 5)) \end{cases}$$

$$B = y((E, 4), (F, 1)) \begin{cases} E = x((1, 4), (6, 3)) \\ F = x((2, 0)) \end{cases}$$





Take-home Message

- 数组的顺序存储结构
- 特殊矩阵的压缩存储
- 稀疏矩阵的表示和操作
- 广义表的定义及表示